

TEMA 6. KVM

Virtualización → Tecnología que permite crear entornos virtuales en un sistema físico. Esto se logra mediante la creación de máquinas virtuales o contenedores.

- Total (Completa) → Simula casi en su totalidad al hardware real.
- Parcial → Simula parcialmente el hardware real.
- Paravirtualización → No se simula ningún ambiente de hardware.
 - Ventajas de la virtualización:
 - Consolidación de servidores para ahorrar espacio y dinero
 - Utilización de recursos de forma eficiente
 - Aislamiento, lo que aumenta la seguridad y la protección contra fallos
 - Flexibilidad, permitiendo alta disponibilidad y escalabilidad

Máquina virtual → Instancia aislada y virtualizada de un sistema operativo que se ejecuta en un servidor físico. Cada VM tiene sus propios recursos.

Hipervisor → Proceso que crea y ejecuta máquinas virtuales permitiendo que el ordenador host preste soporte a las máquinas virtuales.

- Tipo 1 → Ejecución directa en el hardware físico → KVM, Microsoft Hyper-V.
- Tipo 2 → Ejecución en una capa de software sobre el sistema → VirtualBox.

1. DEFINICIÓN

KVM → Kernel Virtual Machine → Arquitectura de virtualización completa que utiliza QEMU para emular diferentes características de Hardware. Es de software libre para implementar virtualización completa en Linux. Fue desarrollado por Qumranet y adquirido por RedHat.

QEMU → Proceso del componente de espacio de usuario. Proporciona soporte de virtualización completo junto a KVM. Actúa como emulador de dispositivo de hardware. Utiliza el comando virt-manager para la interfaz gráfica o virsh para la línea de comandos.

Libvirt → Es el gestor de KVM. Es un toolkit para gestionar plataformas de virtualización.

Libvirtd → Es el servicio o demonio incluido en el paquete libvirt.

2. CARACTERÍSTICAS DE KVM

Seguridad

- Cada VM se implementa como un proceso.
- Utiliza AppArmor, que encierra a un proceso si compromete al resto.

Gestión de memoria

- KVM hereda las características de gestión de memoria de Linux.
- Para compartir páginas de memorias utiliza KSM (Kernel Same-page Merging).

Almacenamiento

- KVM utiliza cualquier tipo de almacenamiento soportado por Linux.
- El formato nativo de KVM es QCOW2 permitiendo el cifrado, la compresión y la realización de snapshots.

Live Migrations

- KVM permite migraciones en “caliente”.
- Las migraciones son transparentes para el usuario.
- KVM permite almacenar el estado de una VM para almacenarlo y reanudarlo.

Drivers

- KVM soporta la virtualización híbrida.
- El hipervisor KVM utiliza el estándar VirtIO, desarrollado por IBM y RedHat junto con la comunidad de Linux.
 - VirtIO
 - Es una interfaz independiente del hipervisor diseñada para el desarrollo de drivers con el objetivo de conseguir una mayor interoperabilidad con los invitados.

Rendimiento y Escalabilidad

- KVM posee los mismo rasgos que Linux.
- KVM soporta hasta 16 CPUs y 256gb de RAM.
- Se han conseguido ratios de 600 VM en un solo servidor físico.

3. INSTALACIÓN DE KVM

1. Comenzamos buscando el número de CPUs que tenemos.
 - **egrep -c “(svm|vmx)” /proc/cpuinfo** → Comando para saber el número de núcleos (debe ser más de 1)
2. Habilitar la virtualización anidada. Hay que habilitar VT-x/AMD-v en VirtualBox.
 - **VboxManage modifyvm <nombre_VM> --nested-hw-virt on**
3. Instalamos los paquete necesario para virtualizar y hacer bridges.
 - **sudo apt install qemu-kvm libvirt-client libvirt-daemon-system bridge-utils libguestfs-tools genisoimage virtinst libosinfo-bin virt-manager**
4. Comprobamos el servicio Libvirtd
 - **sudo systemctl status libvirtd.service**
5. Agregamos nuestro usuario al grupo de admin de KVM.
 - **Sudo adduser <usuario> libvirt**
 - **sudo adduser <usuario> libvirt-qemu**
 - **newgrp libvirt**
 - **newgrp libvirt-qemu**
6. Agregamos qemu al usuario root en /etc/libvirt/qemu.conf.
 - **user = “root”**
 - **group = “root”**
7. Para utilizar MVs con un usuario que no sea root, hay que añadirlo al grupo “**libvirt**”.
8. Comprobaciones
 - **virt-host-validate**
 - **virt-host-validate | grep -i ‘FALLA/ADVERTENCIA’**

3.1 Mecanismos de Conexión QEMU/KVM

Podemos conectarnos tanto de forma local como remota:

- **Local con usuario no privilegiado** → `virsh --connect qemu:///session`.
 - Accede a las MVs del usuario
 - No puede crear conexiones de red → SLIRP
- **Local con usuario privilegiado** → `virsh --connect qemu:///system`.
 - Accede a las MVs de todo el sistema
- **Remoto con usuario privilegiado ssh** → `virsh --connect qemu+ssh://usuario@IP/system`
 - Nos conectamos a un socket linux `/var/run/libvirt/libvirt-sock`

3.2 Aplicaciones para usar Libvirt

- **virsh** → Cliente por línea de comandos.
- **virt-manager** → Aplicación gráfica.
- **virtinst** → Paquete que proporciona los comando `virt-clone`, `virt-install` y `virt-xml` para crear y copiar MVs.
- **virt-viewer** → Permite acceder a la consola gráfica de la MV. Integrado en `virt-manager`.
- **gnome-boxes** → Aplicación gráfica simple para el usuario en local.

4. REDES EN KVM

Existen **dos** tipos de redes en KVM:

- Redes Virtuales (Privadas) → **vmbrx**.
- Redes Puente (Públicas) → **brx**.

4.1 Redes Virtuales de tipo NAT

Es una red privada → **red default**.

- Crea un bridge virtual → `vmbr0` → utiliza la red del host para conectarse con el exterior.
- Utiliza DHCP para las MVs.
- El host también configura el servidor DNS.
- Las MVs pueden conectarse con el exterior, pero el exterior no con ellas.

<network>

<name>default</name> → Nombre de la red

<bridge name="virbr0"/> → Nombre del bridge virtual a utilizar

<forward mode="nat"/> → Indican si la MV accede al exterior (nat, bridge, router u open)

<ip address="192.168.122.1" netmask="255.255.255.0"> → IP del host

<dhcp> → Activamos el servidor DHCP

<range start="192.168.122.2" end="192.168.122.254"/> → Rango de IPs

</dhcp>

</ip>

</network>

4.2 Redes Virtuales Aisladas (Isolated)

Es una red privada sin conectividad al exterior. Se usan para crear una conexión entre el hipervisor y las MVs → virbr1.

```
<network>
  <name>private</name>
  <bridge name="virbr1"/>
  <ip address="192.168.152.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.152.2" end="192.168.152.254"/>
    </dhcp>
  </ip>
</network>
```

4.3 Redes Virtuales Muy Aisladas (Very Isolated)

Es una red en la que el host no está conectado a las MVs. No existe ni servidor DNS ni DHCP. No tienen salida al exterior → se configura de forma estática.

```
<network>
  <name>private</name>
  <bridge name="virbr2"/>
</network>
```

4.4 Redes Virtuales en modo Abierto

Permite que las MVs se comuniquen entre sí dentro de la red privada.

- **Funcionamiento** → Acceden a la red externa de manera directa, compartiendo la misma red física que el host. No existe NAT.
- **Aislamiento** → No existe aislamiento con la red externa. Las MVs tienen direcciones IP visibles en la red externa y pueden ser accesibles desde fuera.

4.5 Redes Puente Conectadas a un Bridge Externo

Bridge Externo → Bridge virtual que estará conectado al router de la red local. El host está conectado a este bridge para tener conexión con el exterior → **br0**.

- **sudo apt install bridge-utils -y** → Paquete que nos permite trabajar con Linux Bridge

4.6 Redes Puente Compartiendo la Interfaz Física del Host

Utiliza conexión **macvtap** → No utiliza un dispositivo bridge para la conexión. Esto no permite la comunicación directa entre el host y los invitados.

5. RED DEFAULT

Es la red virtual que por defecto instala **qemu-kvm** (virbr0). Esta red opera en modo NAT. Para llegar a la red default:

- Menú → Editar → Detalles de la conexión → Redes virtuales → Configuración IPv4
- El servidor DHCP se ejecuta gracias a → dnsmasq-base
- Por defecto, la red default no está iniciada.

Algunos comandos importantes para la red default son:

- **virsh net-list --all** → Muestra las redes activas e inactivas
- **virsh net-start default** → Inicia la red default
- **virsh net-autostart default** → Activa el inicio automático de la red default

Las MVs que se conecten a esta red (por defecto) tendrán una IP dentro del rango que le hemos marcado anteriormente.

5.1 Creación de Redes desde la Red Default

1. Desde /etc/libvirt/qemu/networks:
 - **virsh net-dumpxml default > nombre_red.xml** → Hace un volcado del fichero xml de nuestra red
2. Generamos un nuevo UUID para la nueva red
 - **sudo apt install uuid-runtime**
 - **uuidgen**
3. Copiamos la UUID para introducirla en el fichero xml
4. Generamos una nueva dirección MAC desde /etc/libvirt/qemu/networks para copiarla en el fichero xml
 - **printf 'DE:AD:BE:EF:%02X:%02X\n' \$((RANDOM%256)) \$((RANDOM%256))**
5. La definimos, la activamos y habilitamos la activación automática desde /etc/libvirt/qemu/networks/
 - **virsh net-define <nombre_red.xml>**
 - **virsh net-start <nombre_red>**
 - **virsh net-autostart <nombre_red>**

6. CREACIÓN DE REDES VIRTUALES PUENTE

6.1 Utilizando Entorno Gráfico (Debian / Ubuntu)

1. Agregamos una nueva red pulsando '+’.
2. Seleccionamos el tipo puente o bridge.
3. Establecemos un nombre y agregamos conexión.
4. Seleccionamos tipo de red cableada o ethernet.
5. Clickamos en la tarjeta de red.
6. **sudo systemctl restart networking.service.**
7. **brctl show** → Verificamos que la red esté activación

6.2 Utilizando Entorno No Gráfico (Debian)

Esta es la manera más sencilla ya que solo tenemos que tocar el fichero `/etc/network/interfaces` y agregar:

- **DHCP**
auto <tarjeta_red>
iface <tarjeta_red> inet manual

#Configuración del Puente Externo
auto br0
iface br0 inet dhcp
bridge_ports <tarjeta_red>
bridge_stp off
bridge_fd 0
bridge_maxwait 0
- **ESTÁTICO**
auto <tarjeta_red>
iface <tarjeta_red> inet manual

#Configuración del Puente Externo
auto br1
iface br1 inet static
bridge_ports <tarjeta_red>
bridge_stp off
address 192.168.122.2
netmask 255.255.255.0
network 192.168.122.0
gateway 192.168.122.1
broadcast 192.168.122.255
hwaddress ether <MAC>

Luego solo hay que reiniciar el servicio de red

- **systemctl restart networking**
- **ip a**
- **ip a flush <tarjeta_red>**
- **ip a flush <nombre_puente>**
- **systemctl restart networking**
- O si lo prefieres, ejecutamos `sudo reboot` para reiniciar el sistema.

6.3 Utilizando Entorno No Gráfico (Ubuntu)

Hay que entrar en el fichero `/etc/netplan/01-network-manager-all.yaml` y añadir las siguientes líneas y usar `sudo netplan apply` para guardar los cambios:

bridges:

br0:

dhcp4: yes

interfaces:

- <tarjeta_red>

7. CONEXIÓN DE REDES PUENTE CON EL BRIDGE EXTERNO

Teniendo ya creado el bridge virtual (br0), al que está conectado el host, podemos crear MVs que estén conectadas a ese bridge.

1. Podemos definir una nueva red → virtual_bridge_network.xml

```
<network>  
  <name>virtual_bridge_network</name>  
  <forward mode="bridge"/>  
</network>
```
2. net-define /usr/share/libvirt/network/virtual_bridge_network.xml
3. net-start virtual_bridge_network

8. CONEXIÓN DE REDES PUENTE CON LA INTERFAZ FÍSICA DEL HOST

Usamos la conexión macvtap, que nos permite conectarnos a la red física a través de una interfaz física del host. Es importante recalcar que la interfaz de red utilizada no puede estar conectada a un puente virtual.

1. Definimos la nueva red → physical_bridge_network.xml

```
<network>  
  <name>virtual_bridge_network</name>  
  <forward mode="bridge">  
    <interface dev="<tarjeta_red>">  
  </forward>  
</network>
```
2. virsh -c qemu:///system net-define /usr/share/libvirt/network/physical_bridge_network.xml
3. virsh -c qemu:///system net-start physical_bridge_network

9. CREACIÓN DE REDES VIRTUALES CON VIRT-MANAGER

1. Accedemos a virt-manager
2. Menú → Editar → Detalles de la conexión → Redes virtuales

10. COMANDOS DE REDES

COMANDO	DESCRIPCIÓN
net-autostart	Habilita el inicio automático de una red
net-create	Crea una red de forma temporal desde el XML
net-define	Crea o actualiza una red inactiva de forma persistente desde su XML
net-destroy	Detiene una red
net-dhcp-leases	Muestra información sobre el asignamiento de IP temporal de una red
net-dumpxml	Imprime el XML de una red
net-edit	Edita de forma interactiva el XML de una red
net-event	Muestra el log o eventos sobre una red

net-info	Muestra la información sobre una red
net-list	Muestra un listado con las redes disponibles
net-name	Convierte el UUID de una red a su nombre
net-start	Inicia una red, previamente definida e inactiva
net-undefine	Elimina una red persistente
net-update	Actualiza alguna parte de la configuración de una red
net-uuid	Convierte el nombre de una red a su UUID
net-port-list	Muestra los puertos de una red
net-port-create	Crea un puerto a través del XML de la red
net-port-dumpxml	Imprime la información de un puerto de red en XML
net-port-delete	Elimina el puerto de red especificado
virsh net-list --all	Estado de las redes
virsh net-info default	Información de la red default
iface-list	Listado de las interfaces físicas

11. CREACIÓN DE MV

11.1 Creación de una MV con Virt-Manager

1. Archivo → Nueva máquina virtual.
2. Selección de los medios de instalación → ISO o CDROM.
3. Elegir la opción de arquitectura → x86_64.
4. Seleccionamos la ISO o CDROM desde el botón Explorar.
5. Desactivamos la opción inferior para seleccionar el sistema operativo.
6. Damos la cantidad de memoria RAM y de CPU que queramos → Siempre inferior a la del equipo anfitrión.
7. Introducimos la cantidad de almacenamiento. Los discos virtuales se guardan en /var/lib/libvirt/images.
8. Nombramos la MV y seleccionamos el tipo de red

11.2 Creación de una MV desde la Línea de Comandos

Para la instalación utilizamos el comando **virt-install** al que le tenemos que dar varios parámetros, pero antes debemos tener la imagen ISO en /var/lib/libvirt/iso (crearemos ese directorio):

- **--name** → Nombre de la máquina virtual
- **--virt-type** → Tipo de virtualización
- **--cdrom** → Instalación desde un fichero ISO
- **--os-variant** → Variante del sistema operativo
- **--disk size** → Tamaño del disco → /var/lib/libvirt/images
- **--memory** → Cantidad de memoria RAM
- **--vcpus** → Cantidad de CPU asignadas
- **--network** → Indicamos la interfaz de red
- **--noautoconsole** → El proceso será en segundo plano
- **--vnc** → Permitiremos la conexión desde un cliente

Ejemplo:

```
virt-install --connect qemu:///system --virt-type kvm --name prueba1 --cdrom ~/iso/alpine-virt-3.20.0-x86_64.iso --os-variant alpinelinux3.17 --disk size=4 --memory 1024 --vcpus 1
```

Comandos para comprobaciones:

- **virsh-list** → Máquinas en ejecución
- **virt-viewer <nombre_MV>** → Acceder a la terminal de una máquina
- **virsh shutdown / reboot <nombre_MV>** → Apaga / reinicia la MV
- **virsh dominfo <nombre_MV>** → información sobre la MV
- **virsh nodeinfo** → Datos de nuestro host KVM / QEMU
- **qemu-img resize </ruta_disco/nombre.qcow2> +1G** → Añadimos 1 GB al tamaño del disco
- **virsh start <nombre_MV>** → Iniciar una MV apagada
- **virsh destroy <nombre_MV>** → Forzar el apagado de la MV
- **virsh resume <nombre_MV>** → Continúa con la ejecución de la MV
- **virsh domifaddr <nombre_MV>** → Obtener la dirección IP de la interfaz de red
- **virsh dombklist <nombre_MV>** → Obtener la información de los discos de la MV

11.3 Creación de una MV desde la Línea de Comandos con Fichero XML

Todo lo que envuelve a una MV está definido en formato XML → **virsh dumpxml <nombre_MV>**.

ETIQUETA	DESCRIPCIÓN
<domain>	Comienzo del documento XML indicando el tipo de virtualización
<name>	Nombre de la MV
<currentMemory>	Memoria asignada a la MV
<vcpu>	Número de vCPUs asignadas
<os>	Información sobre la arquitectura de la MV
<boot>	Orden de arranque
<cpu>	Información de CPU
<devices>	Dispositivos hardware que forman parte de la MV
<disk>	Define los discos (qcow2) → bus='virtio'
<interface>	Interfaces de red

Para crear un disco virtual tenemos que utilizar el siguiente comando:

- **virsh vol-create-as iso <nombre>.qcow2 <cantidadGB> --format qcow2**

Para crear la MV utilizamos el siguiente comando:

- **virsh define <nombre.xml>**
- **virsh start <nombre.xml>** → Para iniciarla

12. ALMACENAMIENTO

- **Pools de Almacenamiento** → Fuente de almacenamiento, cantidad de almacenamiento que el administrador del host ha configurado para su uso en MV.
- **Volúmenes** → División de los pools de almacenamiento.

12.1 Tipos de Tecnologías de Almacenamiento

- Tipos de almacenamiento:
 - **Sistema de ficheros** → Permiten el acceso a parte del sistema de fichero
 - **Dispositivos de bloque** → Gestionar de forma completa un dispositivo en bruto.
- **Almacenamiento compartido** → Comparten información entre distintos servidores con QEMU/KVM + libvirt. Permiten la migración en vivo de máquinas virtuales.
- **Snapshots** → Instantáneas para poder guardar el estado de un disco en un determinado momento, dando la posibilidad de volver a ese estado más adelante.
- **Aprovisionamiento ligero** → Proporciona ahorro en espacio ocupado de almacenamiento. Solo guarda la información que la máquina necesite.

12.2 Tipos de Pools de Almacenamiento

- **dir** → Ofrece un directorio del host (sistema de archivo). No es compartido.
 - **raw** → Formato de imagen binaria sencilla. Se ocupa todo el espacio que hayamos indicado al crearla. No soporta snapshots ni aprovisionamiento ligero.
 - **qcow2** → Formato QEMU copy-o-write. Sólo ocupa el espacio que está ocupando los datos. El fichero irá creciendo cuando escribamos en él. Acepta snapshots.
- **logical** → Utiliza LVM (Logical Volume Manager). El pool controlará un grupo de volúmenes lógicos que se crearán para dar soporte a las MVs. No tiene compartición. Acepta snapshots y aprovisionamiento ligera con LVM-Thin.
- **netfs** → Montará un directorio desde un servidor NAS (nfs, cifs, etc.). Tenemos la característica de compartición y de migración en vivo.

12.3 Gestión de Pools de Almacenamiento

COMANDO	DESCRIPCIÓN	COMPLETO
virsh pool-list	Lista los pools existentes	virsh -c qemu:///system pool-list
virsh pool-info default	Información de la pool por defecto	virsh -c qemu:///system pool-info default
virsh pool-dumpxml default	Vuelca la pool default a XML	virsh -c qemu:///system pool-dumpxml default
virsh -c qemu:///system pool-define new-pool-definition.xml	Crea un pool desde un XML	
virsh pool-create	Crea y arranca un pool no persistente	
virsh pool-define	Crea y arranca un pool persistente	
virsh pool-destroy <nombre_pool>	Detiene un pool	virsh -c qemu:///system pool-destroy new-pool
virsh pool-delete	Borra el directorio creado del	virsh -c qemu:///system pool-delete

<nombre_pool>	pool	new-pool
virsh pool-undefine <nombre_pool>	Elimina el pool creado	virsh -c qemu:///system pool-undefine new-pool

12.4 Creación de un Pool

- 1ª Forma: Volcado de un fichero XML en /etc/libvirt/storage
 - **virsh pool-dumpxml default > poolnuevo.xml**
- 2ª Forma: Creación de un pool con el comando mkdir en /var/lib/libvirt
 - **mkdir <nombre_dir>**
 - **virsh pool-define-as <nombre_dir> --type dir --target /var/lib/libvirt/<nombre_dir>**
- 3ª Forma: Utilizando virt-manager
 - **Menú Editar → Detalles de la conexión → Almacenamiento → ‘+’**

En la 1ª y 2ª forma, el pool se crea pero no se arranca:

- **virsh pool-start**

13. DISCOS, VOLÚMENES Y CLONACIONES

Son los medios de almacenamiento que utilizarán las máquinas virtuales. Un pool está formado por varios volúmenes.

Existen dos enfoques para crear los volúmenes:

- **API de libvirt** → Si creamos un pool de tipo dir, creamos un fichero de ISO, si lo creamos en tipo logical, crearíamos un volumen lógico LVM
- **Herramientas específicas** → Crea los medios de almacenamiento y luego refresca el pool para que añada el nuevo volumen.

El formato de imagen qcow2 tiene la característica de aprovisionamiento ligero, es decir, irá creciendo conforme se guarde información en la imagen.

- **virsh vol-list default --details** → Información sobre el espacio ocupado .
- **virsh vol-info <nombre_vol>.qcow2 default** → Información sobre un volumen.
- **virsh vol-dumpxml <nombre_vol>.qcow2 default** → Fichero XML.

Para crear discos con formato **RAW**:

- **virsh vol-create-as default <nombre_disco> <tamaño>**

Para crear discos con formato **QCOW2**:

- **virsh vol-create-as --pool default --format qcow2 <nombre_disco.qcow2> <tamaño>**

13.1 Añadir Discos

Cuando creamos una MV virtual mediante la línea de comandos, tenemos que indicar la ruta en la que se encuentra el disco previamente creado con la opción

- **--disk vol=default/<nombre_vol.qcow2>**

También podemos añadir discos a MV ya **creada** con el siguiente comando:

- **virsh attach-disk <nombre_MV> --source /var/lib/libvirt/images/<nombre_disco.qcow2> <nombre_disco> --driver=qemu --persistent --subdriver qcow2**

13.2 Redimensionar Discos

Hay dos formas de redimensionar un disco:

- **Máquina parada** → `virsh -c qemu:///system vol-resize <nombre_vol.qcow2> <tamaño> --pool <nombre_pool>`
- **Máquina encendida** → `virsh -c qemu:///system blockresize <nombre_MV> /srv/images/<nombre_vol.qcow2> <tamaño>`

Para que los cambios hagan efecto, hay que utilizar los siguiente comandos:

- **umount /mnt**
- **resize2fs**
- **mount /dev/<nombre_disco> /mnt**

13.3 Clonación

El comando **virt-clone** viene incluido en el paquete de virtinst.

Para clonar MV → **virt-clone --original <nombre_MV> --auto-clone** → Esto clonará la MV al completo.

El comando **virt-sysprep** sirve para eliminar los identificadores únicos de cada MV para poder utilizarla como base para crear nuevas MVs. Viene incluido en el paquete **libguestfs-tools**

- **virsh domrename <nombre_MV> <nuevo_nombre_MV>** → Cambia el nombre de las MV.
- **virsh sysprep -d <nombre_imagen_MV> --hostname <hostname> --root password** → Establece hostname o contraseña de los usuarios, etc.
- **virt-customize** → Establece hostname, inyecta claves SSH, instala paquetes, etc.

13.4 Plantillas

Una plantilla es una imagen previamente configurada utilizada para desplegar rápidamente MVs.

- **Clonación completa** → Copia completa de la MV, requiere el mismo espacio que la original.
- **Clonación enlazada** → Utiliza una imagen de la plantilla como imagen base, requiere menos espacio en disco pero no puede ejecutarse sin acceso a la imagen de plantilla base.

14. SNAPSHOTS

Snapshot → Guarda el estado de una MV en un determinado momento. Permite el volver atrás y recuperar esa imagen → **QCOW2** nos permite hacer snapshots.

COMANDO	DESCRIPCIÓN
virsh snapshot-create-as <nombreMV> --name <nombre_snap> --description <descripción> --atomic	Creación de un snapshot
virsh list	MV encendidas
virsh snapshot-list --domain <nombre_MV>	Consultar los snapshots por MV
Virsh snapshot-list --name <nombre_snap>	Consultar los snapshots por nombre de snapshot

virsh snapshot-info <nombre_MV> <nº_snap>	Información detallada de un snapshot
virsh snapshot-revert <nombre_MV> <nombre_snap>	Revertir un snapshot
virsh -c qemu:///system snapshot-dumpxml nom-maq nom-snapsho t	Consulta la definición XML
virsh snapshot-delete nom-maq nom-snapshot	Eliminación de snapshot
virsh snapshot-delete nom-maq número-snapshot	Eliminar un snapshot

- **Snapshots internos** → Se almacenan en el mismo disco duro de la MV.
- **Snapshots externos** → Se crea un disco duro adicional en el que se almacenan los cambios.

15. CONEXIÓN POR SSH

Tenemos que tener el paquete ssh instalado y darle permiso de root a la conexión ssh

- **apt install openssh-server**
- **nano /etc/ssh/sshd_config** → **PermitRootLogin yes**
- **systemctl restart sshd**

Desde el servidor, creamos al usuario prueba

- **adduser prueba**
- **adduser prueba libvirt**
- **adduser prueba libvirt-qemu**

Desde el cliente, instalamos los siguiente paquetes:

- **apt install ssh-askpass**
- **apt install ssh-askpass-gnome**

15.1 Conexión con Entorno Gráfico

Desde el virt-manager → Archivo → Nueva conexión → SSH → Rellenamos los parámetros.

15.2 Conexión sin Entorno Gráfico

- **virsh -c qemu+ssh://usuario_remoto@IP_servidor/system**